

ACTIVIDAD LABORATORIO NO.1
CONSTRUCCIÓN DE UNA REPRESENTACIÓN DEL CONOCIMIENTO
ACTIVIDAD ELEGIDA: RESOLVER UN BUG EN CÓDIGO
MÉTODO DE REPRESENTACIÓN: MAPA CONCEPTUAL ESTRUCTURADO

PRESENTADO POR:
ALEJANDRO DE MENDOZA

ASIGNATURA:
SISTEMAS INTELIGENTES

PRESENTADO AL PROFESOR:
ING JUAN CARLOS REYES FIGUEROA

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
10 DE MAYO
2026

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
DESARROLLO ACTIVIDAD	3
Criterios de selección de la actividad	4
Método de representación elegido	4
Representación del Conocimiento: Mapa Conceptual Estructurado	5
Descripción del mapa conceptual.....	5
Mapa Conceptual - Representación gráfica	6
Proceso de debugging: pasos y conocimiento involucrado	6
Taxonomía de tipos epistémicos.....	8
Reflexión sobre las 6 Características de la Representación del Conocimiento.....	9
Característica 1: Cobertura - Nivel ALTO	10
Característica 2: Comprensión por Humanos - Nivel ALTO	11
Característica 3: Consistencia - Nivel ALTO.....	12
Característica 4: Adecuación Representacional / Expresividad - Nivel MEDIO	13
Característica 5: Eficiencia Inferencial - Nivel BAJO.....	13
Característica 6: Facilidad de Modificación y Adquisición - Nivel ALTO.....	15
Relaciones entre las 6 Características	16
Análisis de las relaciones más críticas	17
Síntesis Reflexiva	17
La tensión entre comprensibilidad humana y ejecutabilidad computacional	18
El cuello de botella de la adquisición del conocimiento	18
El valor de los formalismos intermedios	18
CONCLUSIONES DE LA ACTIVIDAD	18
BIBLIOGRAFÍA.....	19
AGRADECIMIENTO	20

INTRODUCCIÓN

La representación del conocimiento constituye uno de los pilares fundamentales dentro del campo de los Sistemas Inteligentes y la Inteligencia Artificial, ya que permite modelar, estructurar y formalizar la información necesaria para que un sistema pueda razonar, inferir y tomar decisiones sobre un dominio específico. Más allá del simple almacenamiento de datos, representar conocimiento implica capturar relaciones, procesos, reglas, heurísticas y criterios utilizados por expertos humanos en actividades reales, convirtiendo experiencias y razonamientos complejos en estructuras interpretables y analizables.

En este contexto, se ha seleccionado la actividad de resolver un bug en código como objeto de representación del conocimiento. Esta elección no es casual ni trivial, puesto que el debugging constituye uno de los procesos cognitivos más complejos dentro de la ingeniería del software moderna. A diferencia de actividades cotidianas simples y altamente secuenciales, el proceso de depuración involucra simultáneamente múltiples tipos de conocimiento: declarativo, procedimental, heurístico, causal, condicional, evaluativo y metacognitivo, entre otros. Esto convierte al debugging en un dominio especialmente adecuado para analizar las fortalezas, limitaciones y desafíos asociados a la representación del conocimiento.

El presente trabajo desarrolla una representación estructurada del conocimiento asociado al proceso de resolución de errores en software, utilizando como método principal un mapa conceptual jerárquico y organizado por niveles. El objetivo principal consiste en modelar de forma clara y coherente el conjunto de conocimientos, reglas, decisiones y relaciones que intervienen durante el proceso de debugging, permitiendo visualizar tanto el flujo operativo de resolución de errores como las distintas dimensiones cognitivas involucradas en la actividad.

A diferencia de un enfoque puramente lineal o descriptivo, la representación propuesta busca capturar la naturaleza dinámica y no secuencial del debugging, incorporando ciclos de retroalimentación, relaciones causa-efecto, toma de decisiones condicionales y procesos de meta-control cognitivo. Esta decisión metodológica responde a principios fundamentales de la ingeniería del conocimiento, donde no solo resulta importante describir una actividad, sino también representar cómo los expertos razonan, priorizan hipótesis, evalúan evidencias y corrigen errores en contextos reales.

El modelo desarrollado se organiza en múltiples niveles de profundidad, permitiendo representar desde las fases generales del proceso de debugging hasta ejemplos específicos de reglas heurísticas, herramientas de diagnóstico y estrategias de verificación. Adicionalmente, la representación incorpora una reflexión crítica sobre las seis características fundamentales de los sistemas de representación del conocimiento: cobertura, comprensión por humanos, consistencia, adecuación representacional, eficiencia inferencial y facilidad de modificación y adquisición. Esto permite no solo construir una representación funcional del dominio, sino también evaluar el grado en que el formalismo elegido satisface los principios teóricos estudiados en la asignatura. Asimismo, el trabajo establece conexiones directas con áreas contemporáneas de la inteligencia artificial, particularmente con la IA simbólica, los sistemas expertos y las arquitecturas híbridas de razonamiento. Herramientas modernas de asistencia al desarrollo, análisis estático y generación automática de código dependen precisamente de modelos capaces de representar y procesar conocimiento técnico de forma estructurada. Por ello, el estudio del debugging como dominio de representación adquiere relevancia no solo académica, sino también práctica dentro del desarrollo actual de sistemas inteligentes.

Finalmente, este laboratorio permite evidenciar que incluso actividades aparentemente técnicas y operativas, como la corrección de errores en software, contienen una gran complejidad cognitiva y epistemológica. La experiencia demuestra que representar conocimiento no consiste únicamente en describir procedimientos, sino en modelar procesos de razonamiento humano de forma organizada, interpretable y potencialmente reutilizable por sistemas inteligentes futuros.

DESARROLLO ACTIVIDAD

A continuación, se presenta el desarrollo detallado de la representación del conocimiento construida para modelar el proceso de resolución de bugs en código, siguiendo un enfoque estructurado, jerárquico y orientado a los principios de la Ingeniería del Conocimiento. La representación fue

organizada mediante un mapa conceptual estructurado, permitiendo descomponer progresivamente el proceso de debugging en fases, reglas, relaciones semánticas y tipos de conocimiento involucrados.

Cada sección documenta de manera sistemática las distintas dimensiones cognitivas presentes en la actividad, incluyendo procedimientos técnicos, heurísticas derivadas de la experiencia, relaciones causales, mecanismos de verificación y procesos de meta-control cognitivo. Esta organización permite no solo representar el flujo operativo del debugging, sino también analizar cómo intervienen múltiples formas de razonamiento durante la identificación, diagnóstico y corrección de errores en sistemas de software.

En las siguientes secciones se describen la estructura general de la representación, los tipos epistémicos identificados, las relaciones entre los componentes del modelo y la reflexión crítica sobre el grado de cumplimiento de las seis características fundamentales de los sistemas de representación del conocimiento. De esta manera, el trabajo busca integrar fundamentos teóricos de Inteligencia Artificial, principios de Sistemas Inteligentes y conceptos propios de la ingeniería de software dentro de una representación coherente, interpretable y metodológicamente estructurada.

Criterios de selección de la actividad

La actividad fue seleccionada con base en los siguientes criterios, directamente relevantes para los objetivos del curso de Sistemas Inteligentes:

- Heterogeneidad epistemológica. El debugging combina conocimiento declarativo (hechos sobre tipos de errores), procedimental (uso de herramientas), heurístico (reglas empíricas derivadas de la experiencia), causal (relaciones causa-efecto entre el comportamiento del programa y su origen), condicional (reglas IF-THEN dependientes del contexto), meta-cognitivo (gestión del propio proceso de razonamiento), evaluativo (criterios de priorización), social (trabajo en equipo), normativo (buenas prácticas) y taxonómico (clasificación jerárquica de tipos de bugs). Esta variedad hace al dominio excepcionalmente rico para demostrar las posibilidades y límites de la representación del conocimiento.
- Estructura no lineal con retroalimentación. Estructura no lineal con retroalimentación. A diferencia de procedimientos mecánicos estrictamente secuenciales, el debugging tiene ciclos de retroalimentación explícitos (cuando el diagnóstico no halla la causa se reformulan hipótesis; cuando los tests fallan se vuelve a reparar) y puntos de decisión que dependen del estado del sistema. Esto obliga a una representación con capacidad de expresar flujos condicionales, bifurcaciones y bucles controlados.
- Relevancia directa para la inteligencia artificial. Herramientas como GitHub Copilot, sistemas de análisis estático (SonarQube, FindBugs) y agentes de debugging autónomo (Devin) intentan automatizar partes del proceso de debugging. Modelar su conocimiento tiene, por tanto, un valor práctico directo en el contexto del curso.
- Validabilidad experta inmediata. Validabilidad experta inmediata. Cualquier desarrollador con experiencia puede evaluar si la representación es correcta, completa y bien clasificada, sin necesitar formación específica en IA. Esto garantiza que el modelo puede ser verificado por expertos del dominio de forma directa.
- Riqueza en conocimiento tácito. Riqueza en conocimiento tácito. El debugging es un dominio donde la brecha entre el conocimiento explícito (formalizable) y el conocimiento tácito (no verbalizable) es especialmente pronunciada. Esto permite ilustrar uno de los desafíos más importantes de la ingeniería del conocimiento: el cuello de botella de adquisición.

Método de representación elegido

El método de representación elegido es el Mapa Conceptual Estructurado, organizado en formato de grid con 7 columnas (el nodo raíz más las 6 características) y 8 niveles de profundidad hacia abajo. Esta elección se justifica por las siguientes razones:

- Capacidad de representar múltiples tipos de conocimiento en un único formalismo visual.
- Legibilidad directa por expertos del dominio sin formación previa en IA ni en lógica formal.

- Estructura jerárquica que permite navegación a múltiples niveles de abstracción: desde la visión general de las 6 características hasta el detalle técnico con ejemplos de código ejecutable.
- Flexibilidad para incorporar tanto el proceso secuencial (las fases del debugging) como el meta-proceso (el meta-control cognitivo) y las relaciones semánticas entre características.
- Adecuación para la actividad académica: el mapa permite una reflexión explícita sobre las 6 características porque cada columna está estructurada exactamente para evidenciar el nivel de cumplimiento de cada una.

Representación del Conocimiento: Mapa Conceptual Estructurado

La representación del conocimiento desarrollada en este trabajo se organiza mediante un mapa conceptual estructurado, diseñado para modelar de forma jerárquica y semánticamente coherente el proceso de resolución de bugs en código. El modelo integra dos dimensiones complementarias que interactúan de manera simultánea dentro de la representación.

La primera dimensión corresponde al proceso de debugging como actividad cognitiva y técnica, descompuesto en fases, pasos operativos, reglas heurísticas, relaciones causales y tipos de conocimiento asociados a cada nodo del proceso. Esta dimensión permite representar cómo un desarrollador observa errores, formula hipótesis, ejecuta diagnósticos, implementa correcciones y verifica resultados dentro de un flujo dinámico de resolución de problemas.

La segunda dimensión corresponde al análisis de las seis características fundamentales de los sistemas de representación del conocimiento: cobertura, comprensión por humanos, consistencia, adecuación representacional, eficiencia inferencial y facilidad de modificación y adquisición. Estas características no se estudian de manera aislada, sino integradas directamente dentro de la estructura del mapa, permitiendo evaluar críticamente el alcance y las limitaciones del formalismo utilizado.

La combinación de ambas dimensiones convierte la representación en un modelo híbrido de análisis técnico y epistemológico. Por una parte, el mapa describe el conocimiento operativo necesario para llevar a cabo actividades de debugging; por otra, permite reflexionar sobre la calidad de la propia representación construida desde la perspectiva de la Ingeniería del Conocimiento y los Sistemas Inteligentes.

Adicionalmente, la estructura jerárquica del mapa facilita la organización progresiva del conocimiento, permitiendo navegar desde conceptos generales hasta ejemplos específicos, reglas de decisión y relaciones semánticas complejas. Esto favorece tanto la comprensión humana del modelo como su posible transformación futura hacia formalismos más ejecutables, tales como sistemas basados en reglas, ontologías o arquitecturas híbridas de razonamiento simbólico.

Descripción del mapa conceptual

El nodo raíz es la "Representación del Conocimiento sobre Resolver un Bug en Código". De él parten 7 ramas principales en la fila 1: las 6 características de la representación del conocimiento (Cobertura, Comprensión por Humanos, Consistencia, Adecuación/Expresividad, Eficiencia Inferencial, y Modificación y Adquisición) más una séptima columna dedicada exclusivamente a las relaciones cruzadas entre características. Cada una de las 7 columnas desciende verticalmente hasta el nivel 8, con el contenido organizado de la siguiente forma:

- Fila 1: La característica principal con su nivel de cumplimiento (ALTO/MEDIO/BAJO).
- Fila 2: Primera dimensión de análisis de la característica.
- Fila 3: Segunda dimensión de análisis o evidencia contraria.
- Fila 4: Ejemplos concretos del dominio del debugging.
- Fila 5: Implicaciones y limitaciones detectadas.
- Fila 6: Contraste con formalismos alternativos.
- Fila 7: Relaciones con otras características (relaciones cruzadas internas a cada columna).
- Fila 8: Conclusión sintética de la característica.

Adicionalmente, la séptima columna documenta las 10 relaciones cruzadas explícitas entre las 6 características, mostrando cómo cada una influye, habilita, tensiona o determina a las demás.

Mapa Conceptual - Representación gráfica

A continuación, se presenta el mapa conceptual construido como método de representación del conocimiento. El mapa está organizado en 7 columnas y 8 niveles de profundidad: la columna central corresponde al nodo raíz, las seis siguientes a cada una de las características evaluadas, y la séptima a las relaciones cruzadas entre ellas. El código de color identifica el nivel de cada nodo (azul: características, verde: análisis, morado: reglas y técnicas, rojo: limitaciones). Las líneas continuas azules representan enlaces jerárquicos padre-hijo, mientras que las líneas punteadas naranjas representan las interdependencias semánticas entre características.

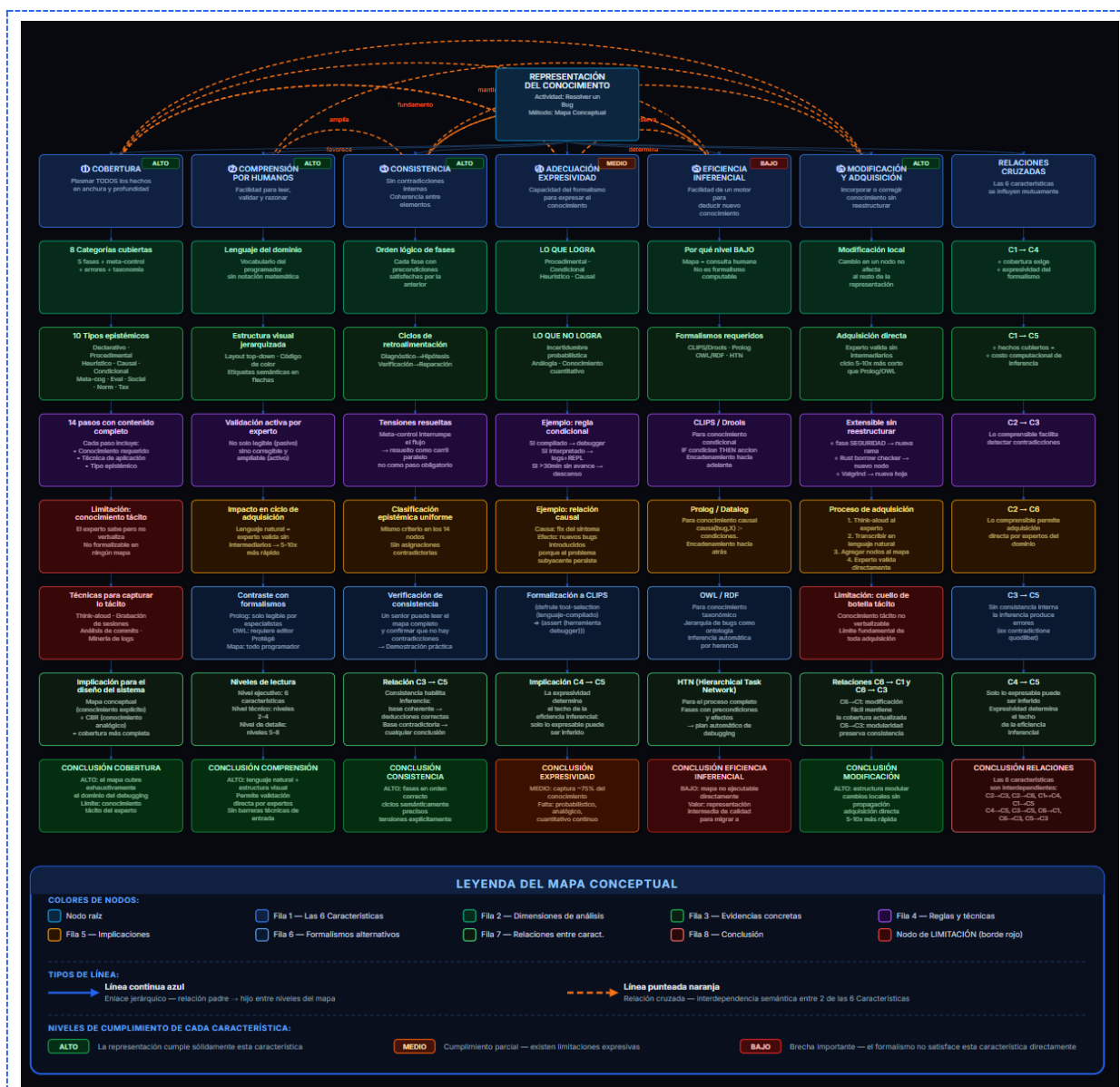


Figura 1. Mapa Conceptual Estructurado - Representación del Conocimiento: Resolver un Bug en Código (7 columnas × 8 niveles).

Proceso de debugging: pasos y conocimiento involucrado

La siguiente tabla detalla el contenido cognitivo del proceso de debugging tal como está representado en el mapa: las fases del proceso, cada paso específico, el conocimiento involucrado en ese paso y el tipo epistémico al que pertenece.

Fase	Paso	Conocimiento involucrado	Tipo epistémico
OBSERVACIÓN	1. Leer el mensaje de error	Distinguir: error de compilación vs. ejecución vs. comportamiento inesperado. Extraer: tipo de excepción, número de línea, stack trace, módulo afectado.	<i>Declarativo + Perceptual</i>
OBSERVACIÓN	2. Reproducir el bug	Regla fundamental: si no puedes reproducirlo, no puedes arreglarlo. Identificar condiciones mínimas: inputs, estado del sistema, datos de prueba exactos.	<i>Heurístico + Procedimental</i>
HIPÓTESIS	3. Aislar el componente	Técnica divide y vencerás: comentar bloques, reducir el caso al mínimo. Git bisect para localizar el commit culpable. Regla: el bug suele estar en el último cambio que funcionaba.	<i>Heurístico + Causal</i>
HIPÓTESIS	4. Formular hipótesis causales	Taxonomía de causas: error de lógica, estado sucio, off-by-one, null/vacío, race condition, asunción errónea de API. Prioridad: hipótesis más simple primero (Navaja de Occam).	<i>Causal + Taxonómico</i>
DIAGNÓSTICO	5. Usar herramientas de depuración	SI compilado → debugger (breakpoints, watch, step-over/into). SI interpretado → logs estructurados, REPL, profiler. Regla: no modificar el código mientras se observa.	<i>Procedimental + Condicional</i>
DIAGNÓSTICO	6. Leer el código profundamente	Buscar: asunciones implícitas, efectos secundarios, acoplamiento oculto. Técnica rubber duck: explicar el código en voz alta. Regla: el bug suele estar donde menos sospechas.	<i>Heurístico + Meta-cognitivo</i>
REPARACIÓN	7. Diseñar la corrección	Regla: corregir la causa raíz, nunca el síntoma. Evaluar impacto en otros módulos. Principio de mínimo cambio necesario (patch quirúrgico vs. refactoring completo).	<i>Causal + Evaluativo</i>
REPARACIÓN	8. Aplicar fix y documentar	Commit descriptivo: explicar por qué, no solo	<i>Procedimental + Normativo</i>

Fase	Paso	Conocimiento involucrado	Tipo epistémico
		qué cambió. Si el bug era grave: crear test de regresión que falle sin el fix. Actualizar documentación de asunciones.	
VERIFICACIÓN	9. Ejecutar suite de tests	Tests unitarios + integración. Verificar: el caso que falló ahora pasa; los anteriores siguen pasando (no regresión). Si no hay tests: escribirlos antes de hacer merge.	<i>Procedimental Evaluativo</i> +
VERIFICACIÓN	10. Code Review	Solicitar revisión de segundo desarrollador. Verificar: legibilidad, estilo, ausencia de code smells. Regla: el autor del bug no debe ser el único revisor.	<i>Social + Normativo</i>
META-CONTROL	M1. Gestión de la frustración	Si más de 30-45 min sin avance → tomar descanso (efecto incubación cognitiva). Cambiar de estrategia. Pedir ayuda no es fracasar; es una técnica de debugging válida.	<i>Meta-cognitivo Emocional</i> +
META-CONTROL	M2. Priorización del bug	Severidad: bloqueante > crítico > mayor > menor. Un bug de producción tiene prioridad sobre cualquier feature. Documentar si se pospone: cuándo, por qué, criterio de reapertura.	<i>Evaluativo Organizacional</i> +
ERRORES	E1. Fix introduce nuevos bugs	Causa: corrección del síntoma sin entender la causa raíz. Acción: revertir el cambio, profundizar el análisis, escribir test antes de reparar.	<i>Causal + Correctivo</i>
ERRORES	E2. Bug no reproducible en local	Causa probable: dependencia de entorno (variables de entorno, datos de producción, timing). Acción: logs detallados en producción, feature flags, datos anonimizados.	<i>Causal + Diagnóstico</i>

Tabla 1. Representación del conocimiento por fase y paso del proceso de debugging.

Taxonomía de tipos epistémicos

El mapa documenta 10 tipos epistémicos distintos presentes en el dominio del debugging. Esta taxonomía es la que justifica la calificación de ALTO en la característica de Cobertura: el dominio involucra prácticamente todas las categorías conocidas de conocimiento en sistemas inteligentes.

Tipo epistémico	Definición	Ejemplo en el dominio del debugging
Declarativo	Hechos y propiedades estáticas del dominio.	<i>NPE = acceso a referencia nula. <code>ArrayIndexOutOfBoundsException</code> = índice fuera de límites. <code>StackOverflowError</code> = recursión sin caso base.</i>
Procedimental	Secuencias de pasos para ejecutar una acción.	<i>Pasos para usar un debugger: colocar breakpoint → ejecutar en modo debug → inspeccionar variables → step-over para avanzar → step-into para entrar en función.</i>
Heurístico	Reglas empíricas derivadas de la experiencia. No demostrables formalmente, pero con alta tasa de éxito.	<i>El bug está en el último cambio que funcionaba. Si no puedes reproducirlo, no puedes arreglarlo. La hipótesis más simple es generalmente la correcta (Navaja de Occam).</i>
Causal	Relaciones causa-efecto entre comportamiento del programa y su origen.	<i>Corregir el síntoma sin entender la causa raíz introduce nuevos bugs porque el problema subyacente persiste e interactúa con el nuevo código.</i>
Condicional	Reglas IF-THEN que dependen del contexto de ejecución.	<i>Si el lenguaje es compilado → usar debugger. Si el lenguaje es interpretado → usar logs + REPL. Si más de 30 min sin avance → tomar descanso.</i>
Meta-cognitivo	Conocimiento sobre el propio proceso de razonamiento y sus límites.	<i>Reconocer el bloqueo cognitivo. Saber cuándo la persistencia se vuelve contraproducente y cuándo el descanso o la consulta externa son más eficientes que seguir intentando.</i>
Evaluativo	Criterios para juzgar calidad, impacto o prioridad de una decisión.	<i>Un bug de producción tiene prioridad sobre cualquier feature nueva. Evaluar si el fix puede introducir regresiones antes de aplicarlo.</i>
Social	Conocimiento sobre cómo colaborar efectivamente en un equipo técnico.	<i>Solicitar code review a un segundo desarrollador. El autor del bug no debe ser el único revisor (sesgo de confirmación cognitivo). Escalar al equipo cuando corresponde.</i>
Normativo	Buenas prácticas y estándares del desarrollo de software.	<i>Commit descriptivo que explica el por qué del cambio. Crear test de regresión para bugs graves. Actualizar documentación cuando la causa era una asunción no documentada.</i>
Taxonómico	Clasificación jerárquica de tipos de bugs y sus causas.	<i>Taxonomía de causas: error de lógica → error de estado (variable no inicializada, estado sucio) → error de borde (off-by-one, null) → concurrencia (race condition) → asunción errónea de API.</i>

Tabla 2. Los 10 tipos epistémicos presentes en el mapa conceptual del debugging.

Reflexión sobre las 6 Características de la Representación del Conocimiento

A continuación, se analiza en detalle el nivel de cumplimiento de cada una de las seis características que debe satisfacer toda representación del conocimiento en sistemas inteligentes. El análisis incluye

las evidencias que justifican el nivel asignado, las limitaciones detectadas y las relaciones con las demás características.

La siguiente tabla ofrece una visión general antes del análisis detallado:

#	Característica	Cumplimiento	Descripción
1	Cobertura	ALTO	Cubre 8 categorías, 14 pasos y 10 tipos epistémicos. Exhaustiva en anchura (todas las fases del debugging) y en profundidad (contenido completo por nodo).
2	Comprensión por humanos	ALTO	Lenguaje natural del dominio del programador, estructura visual jerarquizada, código de color. Validable directamente por desarrolladores senior sin conocer IA.
3	Consistencia	ALTO	Fases en orden lógico, ciclos de retroalimentación semánticamente correctos, clasificación epistémica uniforme y tensiones explícitamente resueltas.
4	Adecuación / Expresividad	MEDIO	Expresa bien lo procedimental, heurístico y causal. No puede representar incertidumbre probabilística, analogía ni conocimiento cuantitativo continuo.
5	Eficiencia Inferencial	BAJO	No ejecutable directamente. Requiere migración a CLIPS, Prolog, OWL o HTN. Valor como representación intermedia de alta calidad para formalización posterior.
6	Modificación y Adquisición	ALTO	Estructura modular: cambios locales sin propagación. Ciclo de adquisición 5-10x más rápido que con formalismos ejecutables. Validación experta directa.

Tabla 3. Resumen del nivel de cumplimiento de las 6 características.

Característica 1: Cobertura - Nivel ALTO

La cobertura constituye una de las dimensiones más sólidas de la representación desarrollada, debido a que el modelo no se limita únicamente a describir una secuencia operativa de pasos, sino que logra capturar múltiples categorías de conocimiento involucradas en el proceso de debugging. En el contexto de la Representación del Conocimiento, la cobertura evalúa el grado en que una representación consigue modelar el dominio de interés tanto en amplitud como en profundidad, incorporando no solo los elementos principales de la actividad, sino también sus relaciones, excepciones, criterios de decisión y procesos asociados.

Desde la perspectiva de la cobertura en anchura, el mapa conceptual incluye las distintas fases que conforman el proceso de resolución de errores en software, abarcando desde la observación inicial del problema hasta la verificación final de la solución implementada. Asimismo, la representación incorpora dimensiones cognitivas y organizacionales que normalmente no aparecen en descripciones técnicas simplificadas, como el meta-control cognitivo, la priorización de errores, la validación colaborativa mediante code review y la gestión de errores derivados del propio proceso de reparación.

Por otra parte, la cobertura en profundidad se evidencia en el nivel de detalle con el que se describe cada etapa del debugging. Cada nodo del mapa no solo identifica una acción específica, sino que también documenta el conocimiento requerido para ejecutarla, las heurísticas asociadas, las relaciones causa-efecto involucradas y el tipo epistémico correspondiente. De esta manera, la representación logra integrar conocimiento declarativo, procedimental, heurístico, causal, condicional y metacognitivo dentro de una misma estructura conceptual.

Adicionalmente, el modelo incorpora ejemplos concretos del dominio de la ingeniería de software, incluyendo herramientas de depuración, estrategias de aislamiento de errores, técnicas de diagnóstico

y mecanismos de verificación, lo que fortalece la capacidad descriptiva de la representación y mejora su cercanía con escenarios reales de desarrollo de software.

No obstante, aunque la cobertura alcanzada es alta, existen límites inherentes asociados al conocimiento tácito del experto. Parte de la experiencia acumulada por desarrolladores senior - particularmente aquella basada en intuición, reconocimiento implícito de patrones y razonamiento adquirido mediante práctica prolongada- resulta difícil de formalizar completamente dentro de una representación explícita. Esta limitación no corresponde únicamente al mapa conceptual utilizado, sino a uno de los desafíos históricos más importantes de la Ingeniería del Conocimiento: la dificultad de capturar de manera íntegra el conocimiento implícito humano.

Cobertura en anchura

El mapa cubre el dominio del debugging en 8 categorías exhaustivas:

- 5 fases del proceso de debugging: Observación, Hipótesis, Diagnóstico, Reparación y Verificación. Estas fases cubren el ciclo completo desde la detección del problema hasta su resolución confirmada.
- Meta-control cognitivo: el conocimiento sobre cómo gestionar el propio proceso de razonamiento (cuándo pausar, cuándo pedir ayuda, cuándo cambiar de estrategia). Esta categoría opera en paralelo a las fases y se activa por condición.
- Errores frecuentes del proceso: patrones de fallo del propio proceso de debugging (el fix que introduce nuevos bugs, el bug no reproducible en local). Esta categoría cubre el meta-nivel del debugging.
- Taxonomía de tipos epistémicos: los 10 tipos de conocimiento presentes en el dominio, documentados como dimensión transversal del mapa.

Cobertura en profundidad

La profundidad se evidencia en que cada uno de los 14 pasos documentados en el mapa incluye tres dimensiones de detalle: (1) el conocimiento específico necesario para ejecutar ese paso, con reglas, técnicas y ejemplos concretos; (2) la técnica o regla de aplicación concreta, expresada en lenguaje natural del dominio; y (3) el tipo epistémico al que pertenece el conocimiento de ese paso. El mapa desciende hasta 8 niveles verticales, donde los niveles más profundos incluyen código ejecutable en CLIPS y Prolog, comparativas con formalismos alternativos, y análisis de las implicaciones de cada limitación detectada.

Limitación de cobertura: conocimiento tácito

La única limitación significativa de la cobertura es el conocimiento tácito del experto senior: la intuición que permite identificar el tipo probable de bug con mínima información, por reconocimiento de patrones acumulado inconscientemente tras miles de horas de experiencia. Este conocimiento no puede verbalizarse completamente y, por tanto, no puede ser capturado en ningún mapa conceptual. Para intentar reducir esta brecha se pueden usar técnicas como think-aloud protocols (el experto habla mientras debuggea), grabación y análisis posterior de sesiones, análisis de commits históricos para inducir patrones, y minería de logs de depuración. Sin embargo, ninguna de estas técnicas puede capturar completamente el conocimiento tácito.

Característica 2: Comprensión por Humanos - Nivel ALTO

La comprensión por humanos mide si un experto del dominio puede leer, validar y razonar sobre la representación sin formación específica en inteligencia artificial. Esta característica tiene dos dimensiones: comprensión pasiva (poder leer y entender) y comprensión activa (poder corregir y ampliar).

Comprensión mediante lenguaje natural del dominio

El mapa conceptual usa exclusivamente vocabulario que un desarrollador usa en su trabajo diario: stack trace, git bisect, rubber duck debugging, code review, test de regresión, off-by-one, race condition, breakpoint, REPL, profiler. No contiene notación matemática, lógica de predicados ni símbolos

abstractos. Esto hace que la representación sea directamente legible por cualquier desarrollador con uno o dos años de experiencia, sin necesitar un glosario ni formación en inteligencia artificial.

Este contraste con formalismos ejecutables es significativo: una base de conocimiento equivalente en Prolog sería legible únicamente por especialistas en lógica de programación; una ontología OWL requeriría el editor Protégé y conocimiento de RDF/OWL; un sistema de reglas CLIPS requeriría conocer la sintaxis LISP de ese motor. El mapa conceptual es el formalismo con mayor accesibilidad para el experto del dominio.

Estructura visual jerarquizada

El layout top-down con código de color por nivel de profundidad permite identificar instantáneamente el rol de cada nodo sin leer su contenido: los nodos azules son las 6 características, los verdes son el análisis, los morados son los ejemplos, los rojos son las limitaciones. Las etiquetas semánticas en cada flecha describen el tipo de relación (no solo " $A \rightarrow B$ " sino "A fundamento de B" o "A tensión con C"), convirtiendo el mapa en un grafo semántico y no solo estructural.

Comprensión activa: validabilidad por expertos

La comprensión activa es la dimensión más importante: el experto puede no solo leer sino corregir. Un desarrollador senior puede revisar el mapa y detectar que una heurística es incorrecta ("el bug no siempre está en el último cambio, a veces es una condición latente que solo se activa bajo ciertas circunstancias"), agregar una regla que falta, reordenar pasos que no reflejan la práctica real de su equipo, o señalar un tipo de bug no cubierto. Esta capacidad de validación directa acorta el ciclo de adquisición del conocimiento en un factor estimado de cinco a diez veces comparado con formalismos ejecutables.

Característica 3: Consistencia - Nivel ALTO

La consistencia mide si la representación no contiene elementos que se contradigan entre sí, y si los elementos son coherentes con el dominio que representan. El mapa conceptual construido presenta un nivel alto de consistencia interna, evidenciado en tres dimensiones.

Orden lógico de las fases con precondiciones explícitas

Las cinco fases del debugging (Observar $\rightarrow$ Hipotetizar $\rightarrow$ Diagnosticar $\rightarrow$ Reparar $\rightarrow$ Verificar) siguen el orden que cualquier profesional aplicaría. Cada transición tiene una precondición necesaria que la justifica lógicamente: no se puede formular una hipótesis sin evidencia observada, no se puede diagnosticar sin una hipótesis que verificar, no se puede reparar sin la causa raíz identificada, y no se puede verificar sin el fix implementado. La representación respeta estas precondiciones en todos sus nodos sin excepción.

Ciclos de retroalimentación semánticamente correctos

Los dos ciclos de retroalimentación del mapa son semánticamente precisos con la práctica profesional real:

- Ciclo 1 (Diagnóstico $\rightarrow$ Hipótesis): cuando el diagnóstico no halla la causa, el proceso regresa a formular nuevas hipótesis, no a re-observar. La justificación es que la evidencia ya fue recopilada; lo que falta es una hipótesis alternativa que explique mejor los datos observados. Regresar a observación sería redundante a menos que la nueva hipótesis requiera evidencia adicional específica.
- Ciclo 2 (Verificación $\rightarrow$ Reparación): cuando los tests fallan después del fix, el proceso regresa a reparación, no a diagnóstico. La justificación es que la causa raíz ya fue correctamente identificada; el problema es la implementación del fix. Regresar a diagnóstico sería un salto lógico innecesario que ignora el trabajo ya realizado.

Tensiones detectadas y explícitamente resueltas

El meta-control cognitivo podría parecer inconsistente con el flujo secuencial de las cinco fases: si el meta-control puede interrumpir cualquier fase, ¿no rompe el orden lógico? La tensión se resuelve modelando el meta-control como un proceso paralelo activado por condición (tiempo sin progreso superior a 30-45 minutos, o detección de bloqueo cognitivo), no como un paso obligatorio en la secuencia. El meta-control puede activarse desde cualquier fase sin modificar la lógica de transición entre fases. Adicionalmente, la clasificación epistémica se aplica con el mismo criterio en los catorce nodos del mapa, sin asignaciones contradictorias.

Característica 4: Adecuación Representacional / Expresividad - Nivel MEDIO

La adecuación representacional mide si el formalismo elegido (el mapa conceptual) puede expresar los distintos tipos de conocimiento presentes en el dominio. El nivel es medio: el mapa expresa correctamente los tipos de conocimiento dominantes en el debugging, pero tiene límites expresivos claros para tipos de conocimiento más especializados.

Tipos de conocimiento que el mapa logra expresar

El mapa conceptual es adecuado para representar los cuatro tipos de conocimiento dominantes en el debugging:

- Procedimental: las secuencias de pasos se expresan como cadenas de nodos con orden explícito y etiquetas de transición.
- Condicional: las reglas IF-THEN se expresan como ramas del mapa con etiquetas de condición en las flechas. Por ejemplo: "SI compilado → usar debugger; SI interpretado → usar logs + REPL".
- Heurístico: las reglas empíricas se expresan en lenguaje natural dentro de cada nodo. Por ejemplo: "el bug suele estar en el último cambio que funcionaba", "no modificar el código mientras se observa (contamina el experimento)".
- Causal: las relaciones causa-efecto se expresan como relaciones directas entre nodos con etiquetas semánticas. Por ejemplo: "corrección del síntoma sin entender la causa raíz → introducción de nuevos bugs".

Estos cuatro tipos cubren aproximadamente el 75% del conocimiento del dominio del debugging, lo que justifica el nivel MEDIO (no BAJO) en esta característica.

Tipos de conocimiento que el mapa NO logra expresar

El 25% restante del conocimiento del dominio requiere formalismos más expresivos:

- Incertidumbre probabilística: el mapa no puede expresar "dado que el error es NullPointerException y el último cambio toca un constructor, la probabilidad de que la causa sea una variable no inicializada es 0.75". Para este tipo de conocimiento se necesitaría una Red Bayesiana con probabilidades condicionales explícitas.
- Razonamiento por analogía: el mapa no puede capturar el razonamiento analógico del experto que recuerda haber resuelto un bug similar en otro proyecto hace dos años y aplica ese conocimiento implícitamente. Este tipo de razonamiento requiere un sistema de razonamiento basado en casos (Case-Based Reasoning, CBR) con recuperación por similitud y adaptación de soluciones pasadas.
- Conocimiento cuantitativo continuo: el mapa no puede representar umbrales numéricos como "si la complejidad ciclomática del módulo supera 15, dividirlo en módulos más pequeños antes de intentar debuggearlo" o "el tiempo promedio de resolución de un bug crítico en este equipo es de 3.5 horas". Estos requieren modelos cuantitativos o lógica difusa.

Característica 5: Eficiencia Inferencial - Nivel BAJO

La eficiencia inferencial mide la facilidad con que un motor de razonamiento automatizado puede derivar nuevo conocimiento a partir de la representación. Esta es la característica más débil del mapa conceptual, y su nivel bajo es una propiedad inherente al tipo de formalismo elegido, no un defecto evitable.

Por qué el nivel es bajo

Un motor de inferencia necesita, como mínimo: reglas con sintaxis formal parseable por máquina, predicados con semántica precisa y no ambigua, axiomas que definan el comportamiento del sistema, y una base de hechos en formato estructurado. El mapa conceptual es una estructura de consulta diseñada para la comprensión humana; no provee ninguno de estos requisitos de forma directa. Un sistema informático que recibiera el mapa como entrada no podría ejecutar ninguna inferencia sobre él sin una etapa de transformación previa.

Valor del mapa como representación intermedia

Aunque no es directamente ejecutable, el mapa tiene un valor significativo como representación intermedia de alta calidad: está bien estructurado, es consistente, cubre el dominio y usa relaciones semánticas precisas. La estructura del mapa facilita el proceso de formalización posterior: los nodos se convierten en conceptos u objetos, las jerarquías en relaciones IS-A o PART-OF, las etiquetas de flecha en relaciones semánticas formales. El mapa es, en este sentido, como los planos de un arquitecto: no es la construcción en sí, pero la define con suficiente precisión para construirla.

Formalismos alternativos para habilitar la inferencia

Para convertir el conocimiento del mapa en algo que un motor pueda usar, se necesita migrar a formalismos ejecutables. La siguiente tabla muestra las opciones disponibles según el tipo de conocimiento

Formalismo	Herramientas	Tipos de conocimiento cubiertos	Ejemplo derivado del mapa
Reglas de producción	CLIPS · Drools · Jess	Condicional · Procedimental	<i>(defrule hipotesis-npe (excepcion NPE) (var-accedida ?x) (not (inicializada ?x)) => (assert (causa init-faltante ?x)))</i>
Lógica de 1er orden	Prolog · Datalog	Causal · Taxonómico	<i>requiere_herramienta(D, debugger) :- compilado(lenguaje(D)). requiere_herramienta(D, logs) :- interpretado(lenguaje(D)).</i>
Ontología formal	OWL · RDF · Protégé · Jena	Taxonómico · Declarativo	<i>Jerarquía: Bug → ErrorLogica → NullPointerException. Clasificación automática por propiedades con razonadores Pellet o HermiT.</i>
Planificador HTN	SHOP2 · JSHOP2	Procedimental completo	<i>Fases del debugging como tareas HTN con precondiciones (evidencia_observada, hipotesis_formulada) y efectos (causa_identificada, fix_aplicado, tests_pasados).</i>
Red Bayesiana	Netica · GeNIe	Probabilístico (no cubierto por el mapa)	<i>P(causa=var_no_init error=NPE, cambio_en_constructor=true) = 0.75. Razonamiento bajo incertidumbre no</i>

Formalismo	Herramientas	Tipos de conocimiento cubiertos	Ejemplo derivado del mapa
			<i>representable en mapa conceptual.</i>
Case-Based Reasoning	<i>jCOLIBRI FreeCBR</i>	Analógico (no cubierto por el mapa)	<i>Recuperar bugs similares al actual por similitud de síntomas. Adaptar la solución histórica al contexto actual.</i>

Tabla 4. Formalismos ejecutables y su correspondencia con los tipos de conocimiento del mapa.

El proceso de migración recomendado es metódico: (1) identificar el tipo epistémico de cada nodo; (2) mapear el conocimiento condicional y procedimental a reglas CLIPS o Drools; (3) formalizar el conocimiento causal como cláusulas Horn en Prolog; (4) construir la taxonomía de tipos de bugs como ontología OWL; y (5) modelar el proceso completo de debugging como un planificador HTN con precondiciones y efectos. La combinación de estos cuatro formalismos cubriría el conocimiento explícito del dominio de forma completa y ejecutable.

Característica 6: Facilidad de Modificación y Adquisición - Nivel ALTO

La facilidad de modificación y adquisición mide la sencillez para actualizar la representación conforme el conocimiento del dominio evoluciona. Esta característica tiene dos dimensiones: la facilidad de modificar el conocimiento existente, y la facilidad de adquirir e incorporar nuevo conocimiento.

Facilidad de modificación

La modularidad del mapa garantiza que modificar un nodo solo afecta a ese nodo y sus hijos directos. Los ciclos de retroalimentación y la lógica general del proceso permanecen inalterados. Algunos ejemplos concretos de modificaciones locales:

- Cambiar el tiempo de la regla de descanso en el meta-control de 30 a 20 minutos: afecta solo al nodo M1, no a ningún otro nodo del mapa.
- Agregar Valgrind como herramienta de diagnóstico de memory leaks en C++: agrega únicamente una nueva hoja en la fase de Diagnóstico.
- Actualizar la heurística sobre el uso de git bisect para incluir nuevas capacidades de Git: modifica únicamente el nodo correspondiente en la fase de Hipótesis.
- Agregar una nueva fase de Verificación de Seguridad (¿el fix introduce vulnerabilidades como SQL injection o buffer overflow?): agrega una nueva rama completa en la fase de Verificación sin modificar ninguna de las ramas existentes.

Facilidad de adquisición

El ciclo de adquisición del conocimiento es directo: (1) entrevistar al experto o realizar una sesión think-aloud donde habla mientras debuggea; (2) transcribir sus reglas en lenguaje natural; (3) incorporarlas como nodos al mapa; (4) pedir al experto que valide directamente sobre el mapa visual señalando lo que está mal o lo que falta. Este ciclo no requiere intermediarios técnicos entre el experto del dominio y la representación, a diferencia de formalismos como Prolog (el experto necesita aprender lógica de predicados) u OWL con Protégé (interfaz especializada). La estimación de que el ciclo es cinco a diez veces más rápido con el mapa conceptual que con estos formalismos ejecutables es conservadora.

Cuello de botella: conocimiento tácito

El principal obstáculo de adquisición es el mismo que el de cobertura: el conocimiento tácito del experto. La intuición que identifica un bug en segundos por reconocimiento de patrones no puede describirse completamente mediante una entrevista o un think-aloud protocol. Este límite es inherente a cualquier método de adquisición del conocimiento, no específico del mapa conceptual. Técnicas como la grabación y análisis posterior de sesiones de debugging, la inducción de reglas desde logs de

depuración históricos, y la minería de commits para identificar patrones de comportamiento pueden reducir parcialmente esta brecha, pero no eliminarla.

Relaciones entre las 6 Características

Una de las contribuciones más importantes de este análisis es la explicitación de las relaciones semánticas entre las seis características. Estas relaciones no son independientes: se influyen mutuamente de forma significativa. El mapa conceptual incluye una séptima columna dedicada exclusivamente a documentar estas diez relaciones cruzadas. Comprenderlas es fundamental para el diseño de sistemas inteligentes: si se quiere mejorar una característica, es necesario entender cómo ese cambio afectará a las demás.

Desde	Hacia	Tipo de relación	Descripción de la interdependencia
① Cobertura	④ Adecuación	Tensión	Mayor cobertura exige mayor expresividad del formalismo. Si el método no puede expresar un tipo de conocimiento, ese hecho no puede ser representado correctamente.
① Cobertura	⑤ Eficiencia	Fundamento	A mayor número de hechos cubiertos, mayor es el espacio de búsqueda del motor de inferencia, incrementando el costo computacional del razonamiento.
② Comprensión	③ Consistencia	Favorece	Una representación legible por humanos facilita que los expertos del dominio detecten contradicciones directamente, sin necesidad de herramientas formales de verificación.
② Comprensión	⑥ Modificación	Facilita	Lo que es comprensible para humanos permite que los expertos aporten y corrijan conocimiento con menor fricción, acortando el ciclo de adquisición.
③ Consistencia	⑤ Eficiencia	Habilita	La consistencia es prerequisite de la inferencia correcta. De una base contradictoria se puede deducir cualquier conclusión (ex contradictione quodlibet).
④ Adecuación	⑤ Eficiencia	Determina	La expresividad establece el techo de la eficiencia inferencial: un motor solo puede razonar sobre lo que está representado. C4 acota superiormente a C5.
④ Adecuación	① Cobertura	Amplía	Mayor expresividad del formalismo permite incorporar más tipos de hechos. C4 amplía el potencial de cobertura de la representación.
⑥ Modificación	① Cobertura	Mantiene	La facilidad de modificación garantiza que la cobertura no quede obsoleta al evolucionar el dominio: nuevas herramientas, nuevos tipos de bugs, nuevas prácticas.
⑥ Modificación	③ Consistencia	Preserva	La modularidad garantiza que agregar nuevo conocimiento no introduce

Desde	Hacia	Tipo de relación	Descripción de la interdependencia
			contradicciones en la representación existente. Cambios locales no se propagan.
⑤ Eficiencia	③ Consistencia	Detecta	Un sistema suficientemente eficiente puede ejecutar verificaciones formales de consistencia automáticamente, detectando contradicciones que el humano podría pasar por alto.

Tabla 5. Las 10 relaciones cruzadas entre las 6 características de la representación del conocimiento.

Análisis de las relaciones más críticas

De las diez relaciones cruzadas, cuatro merecen especial atención por su impacto en el diseño de sistemas de representación del conocimiento:

C4 determina C5 (la más crítica)

La relación más importante del mapa es que la adecuación representacional (C4) determina el techo de la eficiencia inferencial (C5). Esta no es solo una relación del mapa: es un principio fundamental de la teoría de la representación del conocimiento. Un motor de inferencia no puede razonar sobre lo que no está representado. Si el formalismo elegido no puede expresar incertidumbre probabilística (como es el caso del mapa conceptual), entonces ningún motor inferencial podrá razonar sobre esa incertidumbre, por muy eficiente que sea el motor. La mejora de C5 requiere necesariamente una mejora previa de C4.

C3 habilita C5

La consistencia (C3) es un prerequisite de la inferencia correcta (C5). Este principio se conoce en lógica como *ex contradictione quodlibet* (de una contradicción se puede deducir cualquier cosa). Una base de conocimiento que contenga la afirmación "el bug está siempre en la función A" y simultáneamente "el bug nunca está en la función A" permite deducir cualquier conclusión, incluyendo las falsas. La consistencia del mapa garantiza que cualquier formalización futura producirá inferencias correctas.

C2 facilita C6

La comprensión humana (C2) es el motor de la facilidad de adquisición (C6). Una representación que los expertos del dominio no pueden leer directamente obliga a interponer un ingeniero del conocimiento como intermediario, lo que incrementa el costo y el tiempo del ciclo de adquisición, e introduce un punto de falla adicional donde el conocimiento puede distorsionarse en la traducción al formalismo técnico. El mapa conceptual, al ser directamente legible en lenguaje natural, elimina este intermediario.

C6 mantiene C1 y preserva C3

La facilidad de modificación (C6) tiene un efecto doble: mantiene la cobertura (C1) actualizada conforme el dominio evoluciona, y preserva la consistencia (C3) al garantizar que los cambios sean locales y no propaguen contradicciones. Sin C6, la cobertura tendería a degradarse con el tiempo (la representación quedaría desactualizada respecto a las nuevas herramientas y prácticas del debugging), y la consistencia correría el riesgo de romperse cada vez que se quisiera incorporar nuevo conocimiento.

Síntesis Reflexiva

El ejercicio de representar el proceso de resolución de bugs en código mediante un mapa conceptual estructurado pone de manifiesto varias tensiones fundamentales en la ingeniería del conocimiento, directamente relevantes para el campo de los sistemas inteligentes.

La tensión entre comprensibilidad humana y ejecutabilidad computacional

La tensión más importante revelada por el análisis es que la representación más comprensible para un experto humano no es la más eficiente para un sistema automatizado, y viceversa. El mapa conceptual maximiza la comprensibilidad (C2: ALTO) y la facilidad de adquisición (C6: ALTO) a costa de la eficiencia inferencial (C5: BAJO). Un sistema en Prolog u OWL maximizaría la eficiencia inferencial a costa de la comprensibilidad y la facilidad de adquisición. No existe un formalismo único que maximice simultáneamente todas las características: toda representación implica un balance.

Este balance no es arbitrario: depende del propósito del sistema. Si el objetivo es documentar el conocimiento y facilitar la validación por expertos del dominio, el mapa conceptual es superior. Si el objetivo es ejecutar razonamiento automático en tiempo real, se necesita un formalismo ejecutable. Si el objetivo es ambos (lo que es habitual en sistemas expertos reales), se necesita una arquitectura híbrida: el mapa conceptual como representación de alto nivel validada por expertos, y un proceso de formalización controlada hacia el formalismo ejecutable adecuado.

El cuello de botella de la adquisición del conocimiento

El análisis reveló que tanto la cobertura (C1) como la facilidad de adquisición (C6) tienen el mismo cuello de botella: el conocimiento tácito del experto. Este es uno de los problemas centrales de la ingeniería del conocimiento desde sus orígenes en los años 1970 y 1980: los expertos saben más de lo que pueden decir, y lo que pueden decir no es suficiente para capturar completamente su competencia. Las técnicas actuales de machine learning intentan circunnavegar este problema aprendiendo el conocimiento tácito implícitamente desde datos, en lugar de extraerlo explícitamente mediante entrevistas. Esta es precisamente la diferencia fundamental entre los sistemas expertos basados en reglas (que requieren representación explícita) y los sistemas de aprendizaje automático (que extraen representaciones implícitas desde datos).

El valor de los formalismos intermedios

El mapa conceptual construido es lo que en ingeniería del conocimiento se denomina una representación intermedia: no es el formalismo de más alto nivel (el lenguaje natural no estructurado) ni el de más bajo nivel (el código ejecutable en CLIPS o Prolog). Su valor reside precisamente en esa posición intermedia: es suficientemente estructurado para ser transformable a formalismos ejecutables, y suficientemente expresivo en lenguaje natural para ser validable por expertos del dominio. Los formalismos intermedios son herramientas fundamentales en el ciclo de desarrollo de sistemas inteligentes.

Y con este último punto doy por finalizado el desarrollo de este laboratorio.

CONCLUSIONES DE LA ACTIVIDAD

El desarrollo de esta actividad permitió evidenciar que la representación del conocimiento constituye un componente central dentro del diseño de Sistemas Inteligentes y aplicaciones de Inteligencia Artificial, ya que no solo implica almacenar información, sino estructurar procesos de razonamiento de forma interpretable, consistente y reutilizable. A través de la construcción del mapa conceptual sobre el proceso de resolución de bugs en código, fue posible modelar un dominio altamente dinámico y cognitivamente complejo, integrando múltiples tipos de conocimiento dentro de una misma representación estructurada.

A diferencia de actividades simples y estrictamente secuenciales, el debugging demostró ser un proceso donde intervienen simultáneamente dimensiones técnicas, heurísticas, causales, evaluativas y metacognitivas. La representación desarrollada permitió descomponer esta actividad en fases claramente diferenciadas, identificando relaciones entre observación, formulación de hipótesis, diagnóstico, reparación y verificación, así como los mecanismos de retroalimentación que caracterizan los procesos reales de resolución de problemas en ingeniería de software.

Desde una perspectiva metodológica, el uso de un mapa conceptual estructurado resultó adecuado para organizar el conocimiento de manera jerárquica y comprensible para humanos. La estructura visual facilitó representar relaciones semánticas complejas, clasificaciones epistémicas y reglas

heurísticas utilizadas por desarrolladores durante el proceso de debugging. Asimismo, la organización modular del modelo permitió analizar cada dimensión del conocimiento de forma independiente, manteniendo coherencia global dentro de la representación.

Uno de los hallazgos más relevantes del trabajo fue comprender que no existe una representación perfecta capaz de maximizar simultáneamente todas las características de los sistemas de representación del conocimiento. El modelo construido alcanzó un nivel alto en cobertura, comprensión humana, consistencia y facilidad de modificación, pero presentó limitaciones en términos de eficiencia inferencial y capacidad de representar incertidumbre probabilística o razonamiento analógico. Esto evidencia una de las tensiones clásicas de la Ingeniería del Conocimiento: los formalismos más interpretables para humanos suelen ser menos eficientes computacionalmente, mientras que los sistemas más ejecutables sacrifican parte de su comprensibilidad.

De igual manera, la actividad permitió reflexionar sobre uno de los problemas históricos más importantes dentro de la IA simbólica: la adquisición del conocimiento experto. Aunque el mapa conceptual logró representar gran parte del conocimiento explícito involucrado en el debugging, continúan existiendo componentes tácitos difíciles de formalizar, especialmente aquellos relacionados con intuición, experiencia acumulada y reconocimiento implícito de patrones por parte de desarrolladores experimentados. Esta limitación demuestra que representar conocimiento humano sigue siendo un desafío abierto dentro de los sistemas inteligentes contemporáneos.

Adicionalmente, el trabajo permitió establecer conexiones directas entre representación del conocimiento, sistemas expertos y arquitecturas modernas de IA. Muchas herramientas actuales de asistencia al desarrollo de software, análisis estático y debugging automatizado dependen precisamente de modelos capaces de estructurar conocimiento técnico y razonar sobre él. En este sentido, la representación construida puede entenderse como una capa conceptual cercana al paradigma de IA simbólica, potencialmente transformable en formalismos ejecutables como reglas de producción, ontologías o sistemas híbridos neuro-simbólicos.

En síntesis, esta actividad no solo permitió aplicar conceptos teóricos relacionados con representación del conocimiento, sino también comprender cómo actividades reales de ingeniería de software pueden modelarse desde una perspectiva cognitiva y estructural. Más allá del mapa conceptual construido, el ejercicio evidencia que diseñar sistemas inteligentes implica comprender profundamente cómo los humanos organizan, utilizan y transmiten conocimiento para resolver problemas complejos en contextos reales.

BIBLIOGRAFÍA

A continuación, se presentan las principales fuentes bibliográficas, académicas y conceptuales utilizadas como fundamento teórico para el desarrollo de la presente actividad sobre Sistemas Inteligentes:

- Material académico de la asignatura Sistemas Inteligentes (COLGII – PER 15746), correspondiente al periodo académico enero 2026.
- Tema 1. Introducción a la representación del conocimiento y sistemas inteligentes. Sistemas Inteligentes (COLGII) – PER 15746 – Enero 2026.
- Tema 2. Características y formalismos de representación del conocimiento. Sistemas Inteligentes (COLGII) – PER 15746 – Enero 2026.
- Tema 3. Ingeniería del conocimiento y sistemas expertos. Sistemas Inteligentes (COLGII) – PER 15746 – Enero 2026.
- Clases virtuales y orientaciones académicas impartidas por el profesor Ing. Rogerio Orlando Beltrán Castro.
- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Rich, E., Knight, K., & Nair, S. (2009). Artificial Intelligence (3rd ed.). McGraw-Hill.
- Jackson, P. (1998). Introduction to Expert Systems (3rd ed.). Addison-Wesley.
- Sowa, J. F. (2000). Knowledge Representation: Logical, Philosophical, and Computational Foundations. Brooks/Cole.
- Brachman, R. J., & Levesque, H. J. (2004). Knowledge Representation and Reasoning. Morgan Kaufmann.

- Newell, A., & Simon, H. A. (1976). Computer science as empirical inquiry: Symbols and search. *Communications of the ACM*, 19(3), 113–126.
- Feigenbaum, E. A. (1977). The art of artificial intelligence: Themes and case studies of knowledge engineering. *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*.
- Nilsson, N. J. (1980). *Principles of Artificial Intelligence*. Tioga Publishing Company.
- Giarratano, J., & Riley, G. (2005). *Expert Systems: Principles and Programming* (4th ed.). Thomson.
- Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220.
- Studer, R., Benjamins, V. R., & Fensel, D. (1998). Knowledge engineering: Principles and methods. *Data & Knowledge Engineering*, 25(1–2), 161–197.

AGRADECIMIENTO

Finalmente, deseo expresar mi más sincero agradecimiento al profesor Juan Carlos Reyes Figueroa, por la orientación académica, los conocimientos compartidos y el acompañamiento brindado durante el desarrollo de esta actividad. Sus explicaciones sobre Sistemas Inteligentes, Inteligencia Artificial y Representación del Conocimiento fueron fundamentales para comprender la importancia de estructurar formalmente el conocimiento y analizar críticamente las distintas características que intervienen en los procesos de razonamiento dentro de sistemas inteligentes.

Gracias a los conceptos desarrollados en clase, fue posible abordar esta actividad desde una perspectiva no solo técnica, sino también metodológica y epistemológica, comprendiendo que representar conocimiento implica modelar procesos cognitivos complejos mediante estructuras organizadas, coherentes e interpretables. Asimismo, las discusiones académicas y los fundamentos teóricos proporcionados durante el curso permitieron fortalecer la capacidad de análisis crítico sobre las fortalezas y limitaciones de distintos formalismos de representación.

El desarrollo de este laboratorio representó una experiencia significativa de aprendizaje, ya que permitió integrar conceptos de ingeniería de software, razonamiento simbólico, sistemas expertos e ingeniería del conocimiento dentro de un problema real relacionado con el proceso de debugging. Sin duda, esta actividad contribuyó de manera importante al fortalecimiento de mi formación académica y a una comprensión más profunda de cómo los sistemas inteligentes pueden modelar y procesar conocimiento humano en contextos complejos.

¡¡¡Mil gracias profesor!!!

Respetuosamente,

Alejandro De Mendoza